

NETWORK PROGRAMMING LABORATORY

29 June 2026

In this exam you will build a small local-area-network discovery tool. The program scans an IPv4 subnet and tries to identify which hosts are alive. Instead of sending ICMP Echo Requests, the program sends UDP datagrams to hosts in the subnet and observes the ICMP errors generated in response.

The program must send one UDP probe to each usable IPv4 address in the selected subnet. The UDP destination port is expected to be closed on all targets. Therefore, hosts that are reachable should reply with ICMP *Destination Unreachable, Port Unreachable* messages.

The program should use packet capture through `pcap` to observe incoming ICMP packets and should extract the source IPv4 address and the Ethernet source MAC address of each responding host.

For each discovered host, the program must print:

- the IPv4 address;
- the corresponding MAC address;
- optionally, the reverse DNS name, if reverse DNS resolution is enabled.

Command Line

The program must be invoked as follows:

```
sudo host-discovery -n <subnet> -i <interface> [options]
```

Example:

```
sudo host-discovery -n 192.168.1.0/24 -i eth0 --rdns
```

This means that the program scans the subnet `192.168.1.0/24`, while capturing packets on interface `eth0`. If `--rdns` is present, the program should also try to resolve discovered IPv4 addresses to host names.

The following command-line options should be supported:

Option	Meaning
<code>-n, --net</code>	IPv4 subnet to scan, for example <code>192.168.1.0/24</code> .
<code>-i, --iface</code>	Network interface used for packet capture.
<code>-p, --port</code>	UDP destination port used for probes. It may have a default value.
<code>-t, --timeout-s</code>	Maximum capture time in seconds. It may have a default value.
<code>--delay-us</code>	Delay between consecutive UDP probes, in microseconds. It may have a default value.
<code>--rdns</code>	Enable optional reverse DNS lookup.
<code>-v</code>	Increase verbosity. Multiple occurrences should increase the amount of debugging output.

Sending UDP Probes

The program must enumerate all usable host addresses in the subnet. To this aim, the `ipnet` crate can be used to parse and iterate over IPv4 subnets. In particular, if the command-line argument is stored as an `Ipv4Net`, the usable host addresses in the subnet can be collected with:

```
let targets: Vec<Ipv4Addr> = cli.net.hosts().collect();
```

The method `hosts()` returns an iterator over the host addresses of the network, excluding the network and broadcast addresses when applicable.

For each target address, it must send one UDP datagram to the chosen UDP destination port. The UDP payload may contain arbitrary data. The program may introduce a small configurable delay between consecutive probes in order to avoid sending all packets at once.

Packet Capture

The program must open a packet capture on the selected interface using the `pcap` crate. A suitable BPF filter is:

```
ip and dst host <local_ip> and icmp
```

where `<local_ip>` is the IPv4 address used by the UDP socket to send the probes. To obtain the value of `<local_ip>`, you can use the helper function `get_ipv4_addr_from_udp_socket` provided by the `utils` library:

```
pub fn get_ipv4_addr_from_udp_socket(  
    socket: &std::net::UdpSocket  
) -> Result<String>
```

This function returns the IPv4 address used by the UDP socket to send the probes. The program should run the capture in a separate thread, or otherwise arrange the code so that packet capture and UDP transmission can overlap. The capture loop terminates when:

- all targets have replied; or
- the configured timeout has expired.

ICMP Port Unreachable

The program must recognize ICMP Destination Unreachable messages. For IPv4, an ICMP Port Unreachable message has:

`type = 3,      code = 3`

When such a message is received, the IPv4 source address of the outer IPv4 packet identifies the host that generated the error. The Ethernet source MAC address of the captured frame should be saved as the MAC address associated with that responding host.

The basic ICMP header format is shown in Figure 1.

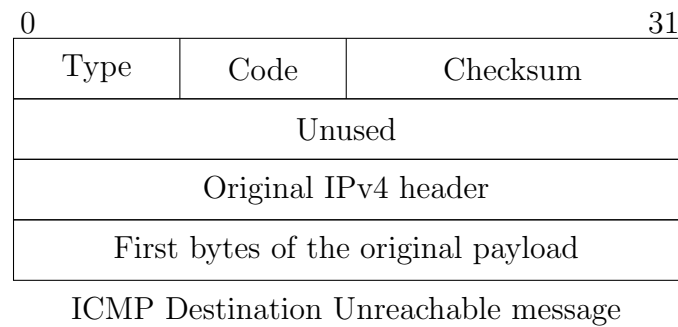


Figure 1: ICMP Destination Unreachable header and embedded original packet data.

Reverse DNS

Reverse DNS lookup is optional and must be controlled by the command-line flag `--rdns`. If the flag is not present, the program must not perform reverse DNS lookups and must not print the host name field. If the flag is present, the program should try to resolve each discovered IPv4 address to a host name. If the lookup fails, the program may print **Unknown**.

To resolve the hostname associated with a discovered IPv4 address, you can use the `dns-lookup` crate. In particular, the function:

```
pub fn lookup_addr(addr: &std::net::IpAddr) -> std::io::Result<String>
```

can be called on the discovered address after converting it to `std::net::IpAddr`. If the lookup succeeds, the returned string can be used as the host name; if it fails, the program may print **Unknown** or omit the name from the output.

Output

For every discovered host, the program should print a line similar to:

```
Host: 192.168.1.12    MAC: a4:83:e7:11:22:33
Host: 192.168.1.25    MAC: 30:9c:23:44:55:66
```

If reverse DNS is enabled, the output should include the resolved host name:

```
Host: 192.168.1.12    MAC: a4:83:e7:11:22:33    Name: laptop.local
Host: 192.168.1.25    MAC: 30:9c:23:44:55:66    Name: Unknown
```

The program may also print verbose diagnostic messages when `-v`, `-vv`, or `-vvv` is used.